

CS-468: Software Quality Assurance & Testing

Structure-Based (White-Box) Testing — Exam Notes

Covers Week 10 (Sessions 1 & 2) and Week 11 (Sessions 1 & 2)

1. Structure-Based (White-Box) Testing — Why It Exists

Structure-based testing looks at the internal structure of the code itself, rather than just its specification. It serves two jobs:

- **Coverage measurement** — checking how much of the code your existing tests (usually written from specification-based techniques) actually exercise.
- **Structural test design** — writing new tests specifically aimed at pushing coverage higher, into parts of the code nothing has touched yet.

So the usual order is: write tests from the specification first → measure how much of the code they hit (coverage) → design extra structural tests to fill the gaps. Because these new tests are built by looking directly at the code, they tend to be different from the specification-based tests, giving the test suite more breadth. If you hit 100% on any coverage measure, you know every part of the software has been touched from that measure's point of view.

2. Test Coverage — The General Idea

Test coverage measures, in some specific and countable way, how much testing a set of tests has actually performed. The rule is simple: if you can count a set of “things” and tell whether each one has been exercised by a test, you can measure coverage of that thing.

$$\text{Coverage} = (\text{Number of coverage items exercised} \div \text{Total number of coverage items}) \times 100\%$$

A “coverage item” is whatever you've chosen to count — it could be statements, decisions, branches, or something else. This general formula is the parent formula that Statement Coverage and Decision Coverage (below) are both built from.

The Danger of Coverage Measures (frequently tested idea)

- 100% coverage does NOT mean 100% tested. It only tells you that every counted item was touched at least once — not that every possible bug was found.
- Coverage measures only one dimension of testing. Two test cases can hit exactly the same statements/decisions, yet one might use input data that reveals a bug the other completely misses.
- Coverage only measures code that has been written. It cannot tell you anything about code that is missing.
 - If a specified function was never implemented, *specification-based testing* will catch that gap.
 - If a function was left out of the specification entirely, *experience-based testing* may catch it.
 - Structure-based techniques can only look at the structure that already exists in the code — they are blind to anything that was never written.

3. Statement Coverage

$$\text{Statement Coverage} = (\text{Number of statements exercised} \div \text{Total number of statements}) \times 100\%$$

Example 1 — one test is enough

Code:

```
READ A READ B IF A > B THEN C = 0 ENDIF
```

To reach 100% statement coverage here, just ONE test case is needed — any input where A is greater than B, e.g. A = 12, B = 10. That makes the IF condition true, so “C = 0” executes and every line in the snippet has now run at least once. Notice that we deliberately chose input values to force full coverage — this is structural test design in action.

Example 2 — tracking coverage line by line

Code:

```
1 READ A 2 READ B 3 C = A + 2*B 4 IF C > 50 THEN 5 PRINT large C 6 ENDIF
```

A	B	C	Lines Covered	Statements Exercised	Coverage %
2	3	8	1, 2, 3, 4 & 6	5	83%
0	25	50	1, 2, 3, 4 & 6	5	83%
47	1	49	1, 2, 3, 4 & 6	5	83%
20	25	70	1, 2, 3, 4, 5 & 6	6	100%

Takeaway: as long as C stays 50 or below, line 5 (“PRINT large C”) never runs, so coverage is stuck at 83% (5 out of 6 lines). You need at least one test where C > 50 to force line 5 to execute and reach 100%.

4. Decision Coverage

A decision is any point with two or more possible outcomes: an IF statement, a loop control statement (DO-WHILE, REPEAT-UNTIL), or a CASE statement.

- IF statement → exits either TRUE or FALSE, depending on the condition.
- Loop control statement → the outcome is either “enter the loop body” or “don't” — again, effectively True/False.

$$\text{Decision Coverage} = (\text{Number of decision outcomes exercised} \div \text{Total number of decision outcomes}) \times 100\%$$

Worked Example

Code:

```
1 READ A 2 READ B 3 C = A - 2*B 4 IF C < 0 THEN 5 PRINT "C negative" 6 ENDIF
```

Start with one test that already gives 100% statement coverage: $A = 20, B = 15 \rightarrow C = 20 - 30 = -10$. Since $C < 0$ is TRUE, the print statement runs and every line in the snippet executes. So statement coverage = 100% already.

But look closer at the IF: only the TRUE branch has actually been exercised. The FALSE branch (skipping straight to ENDIF without printing) has never run — on a control-flow diagram this shows up as a dotted, untested line. So decision coverage here is only 50% (1 of 2 outcomes), even though statement coverage is 100%.

To reach 100% decision coverage, add a second test that forces C to be 0 or positive, e.g. $A = 10, B = 2 \rightarrow C = 10 - 4 = 6 \rightarrow C < 0$ is FALSE. Now both the TRUE and FALSE outcomes have been driven at least once.

1. Test 1: $A = 20, B = 15 \rightarrow C < 0$ is TRUE
2. Test 2: $A = 10, B = 2 \rightarrow C < 0$ is FALSE

Together these two tests give 100% decision coverage. (Any other values that trigger True/False would work equally well — the specific numbers don't matter, only which branch they take.)

Exam insight: Decision coverage is a stricter (stronger) test than statement coverage. 100% decision coverage always guarantees 100% statement coverage — but 100% statement coverage does NOT guarantee 100% decision coverage (exactly as seen above, where one test gave full statement coverage but only 50% decision coverage).

5. Loops in Statement / Decision Coverage

The same repeating logic can be written as a WHILE loop, a DO-WHILE loop, or a FOR loop. All three are logically equivalent (count from 15 up to 20) but their coverage behaviour differs because of when the condition is checked:

Loop Type	Condition Checked	Body Guaranteed to Run?
WHILE (pre-test)	Before each pass	No — could run 0 times if condition starts False
FOR (pre-test)	Before each pass	No — could run 0 times
DO-WHILE (post-test)	After each pass	Yes — always runs at least once

Sample code (from slides):

```
WHILE LOOP 1 X=15 2 COUNT=0 3 WHILE X<20 4   COUNT=COUNT+1 5   X=X+1 6 END WHILE
```

```
DO WHILE LOOP 1 X=15 2 COUNT=0 3 DO 4   COUNT=COUNT+1 5   X=X+1 6 WHILE X<20 7 END WHILE
```

```
FOR LOOP 1 COUNT=0 2 FOR X=15 TO 20 3   COUNT=COUNT+1 4   X=X+1 5 END FOR
```

Exam tip: for a pre-test loop (WHILE/FOR), 100% decision coverage needs one test where the loop condition is True at least once (body runs) and one where it's False the very first time it's checked (body never runs, 0 iterations). For a post-test loop (DO-WHILE), the body always runs at least once by design, so you only need to worry about testing the exit condition becoming True vs staying False across iterations.

6. Case Study — Airline Upgrade / Gold Card

Scenario: a passenger flying economy may get upgraded to business class — especially if they hold a gold card in the airline's frequent-flyer programme. If they don't hold a gold card, there's a chance they get bumped off the flight if economy is full and they check in late. Each box in the flow chart is a numbered statement.

Flow logic (statement numbers as in the original flow chart):

- 1: gold card? → Yes → 2 | No → 6
- 2: business class full? → Yes → 3 | No → 4
- 3: economy → 5 (boarding card)
- 4: upgrade → 5 (boarding card)
- 5: boarding card [end of this branch]
- 6: economy full? → Yes → 8 | No → 7
- 7: economy → 5 (boarding card)
- 8: business class full? → Yes → 10 | No → 9
- 9: upgrade → 5 (boarding card)
- 10: bump off flight [end of this branch]

There are 10 numbered statements in total, and 4 decision points: boxes 1, 2, 6 and 8 (each with a Yes/No outcome).

(a) Test Data for Maximum Decision Coverage

To exercise every Yes/No outcome at every one of the 4 decision points, you need 5 test cases:

Card Type	Business Class Status	Economy Class Status	Path Exercised
Gold	Not Full	Don't Care	1, 2, 4 & 5
Gold	Full	Not Full	1, 2, 3 & 5
Not Gold	Don't Care	Not Full	1, 6, 7 & 5
Not Gold	Not Full	Full	1, 6, 8, 9 & 5
Not Gold	Full	Full	1, 6, 8 & 10

These 5 tests between them drive every decision box (1, 2, 6, 8) down both its Yes and No paths at least once — that's 100% (maximum) decision coverage.

(b) Statement Coverage of Three Specific Tests

Question: if only these three tests are run, what statement coverage does each achieve?

3. Test 1: Gold card holder who gets upgraded to business class → path 1, 2, 4 & 5
4. Test 2: Non-gold card holder who stays in economy → path 1, 6, 7 & 5
5. Test 3: A person who is bumped from the flight → path 1, 6, 8 & 10

Statement Coverage = (Total statements exercised ÷ Total statements) × 100 Total number of statements = 10

Test	Statements Exercised	Total Exercised	Statement Coverage
1	1, 2, 4 & 5	4	$4/10 \times 100 = 40\%$
2	1, 6, 7 & 5	4	$4/10 \times 100 = 40\%$
3	1, 6, 8 & 10	4	$4/10 \times 100 = 40\%$

Each individual test only reaches 40% statement coverage on its own. (If you ran all three together, the combined unique statements would be 1, 2, 4, 5, 6, 7, 8 & 10 = 8 out of 10 = 80% — worth knowing, but the exam question here asks for each test's own coverage, which is 40% each.)

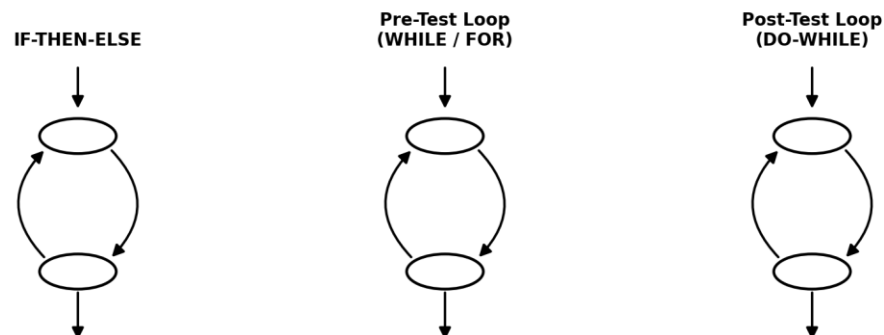
7. Control Flow Graphs (CFG)

A control flow graph represents the decision points and flow of control in a piece of code — essentially a flow chart that ONLY shows decisions, ignoring purely sequential statements (assignments, reads, prints, etc. that don't affect flow).

- **Node** — any point where flow of control can be modified (a decision point), or where a control structure returns to the main flow (e.g. END WHILE, ENDIF).
- **Edge** — a line connecting two nodes.
- **Region** — the closed area enclosed by a collection of nodes and edges.

Standard Subgraph Patterns

Three building blocks come up again and again — learn to recognise and draw these shapes:



If-Then-Else, Pre-Test Loop (WHILE/FOR), and Post-Test Loop (DO-WHILE) subgraph patterns.

- **If-Then-Else:** entry flows into a decision node, splits into two branches, and both branches rejoin at a single node before continuing.
- **Pre-Test Loop (WHILE/FOR):** the condition is checked first — flow can go straight through without ever entering the loop body.
- **Post-Test Loop (DO-WHILE):** the body always runs first, and the condition is checked afterward to decide whether to loop again.

Any chunk of code, however complex, can be represented by combining copies of these three basic subgraphs.

Steps to Draw a Control Flow Graph

6. Analyse the code and identify all control structures — every statement that can change the flow of control — ignoring purely sequential statements.
7. Add one node for each decision (branching) statement you found.
8. Expand each node by substituting in the matching subgraph pattern for that structure (if-then-else, pre-test loop, or post-test loop).

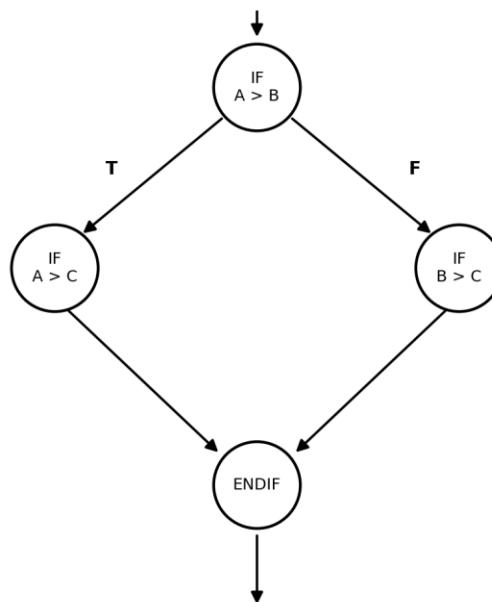
Worked Example — “MaxAndMean” Program

The program reads three integers A, B, C, calculates their mean, then works out the maximum using nested IF statements:

```
If A > B Then      If A > C Then Maximum = A Else Maximum = C Endif Else      If B > C Then Maximum = B Else Maximum = C Endif Endif
```

Step 1 identifies the control structures and ignores the sequential lines (Read A, Read B, Read C, Mean calculation, Print statements): the decisions are “IF A > B”, and nested inside it, “IF A > C” (True branch) and “IF B > C” (False branch).

Steps 2 and 3 add a node for each decision and expand them using the if-then-else subgraph, giving the following control flow graph:



Control flow graph for MaxAndMean — both nested IFs converge on the same final ENDIF node.

Reading tip: whichever branch of “IF A > B” you take (True or False), you immediately land inside a second nested IF — and both nested IFs eventually converge back into the very same ENDIF node at the bottom.

8. Hybrid Flow Graph

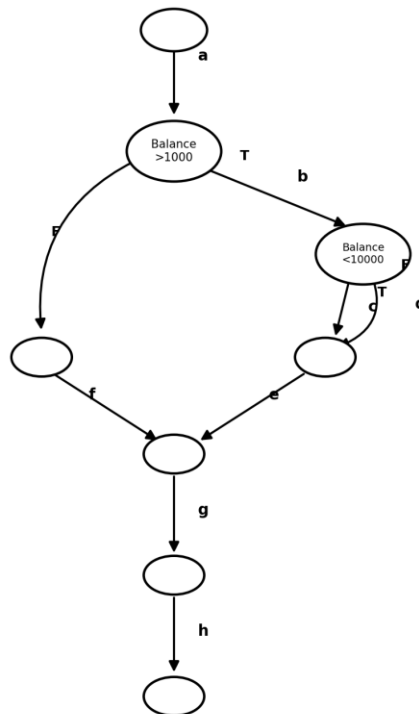
Flow charts, control flow graphs, and hybrid flow graphs all convey the same underlying information — but one format is sometimes more useful than another depending on what you're trying to see. The hybrid flow graph is a useful combination of the control flow graph and the traditional flow chart.

To make it even more useful, we label every edge with a letter. A path through the code then becomes simply a sequence of these labels — for example, 'abeh' describes one particular route through the program from start to end.

Worked Example — “BestInterest” Program

Program logic: Base Rate = 0.035, Interest = Base Rate, then Read Balance. If Balance > 1000, add 0.005 to Interest, then check if Balance < 10000 — if so add another 0.005, otherwise add 0.010. Finally, Balance = Balance × (1 + Interest).

Question: how many test cases are needed for 100% statement coverage, and what should they be?



Hybrid flow graph for BestInterest, with edges labelled a–h. Paths 'abcegh' and 'abdfgh' each give a distinct route through the program.

It's clear from the graph that the left branch (Balance ≤ £1,000) doesn't need to be exercised at all — that path just skips through without adding any interest. But there are two genuinely different paths inside the “Balance > 1000” branch that both need to be tested: Balance between £1,000 and £10,000, and Balance above £10,000.

So two test cases are enough for 100% statement coverage, e.g. Balance = £5,000 and Balance = £20,000.

Same answer, described using path labels: you need to exercise path 'abcegh' (Balance between £1,000 and £10,000) and path 'abdfgh' (Balance above £10,000).

9. One test with Balance between £1,000 and £10,000 (e.g. £5,000 or £8,000) → follows path 'abcegh'.
10. One test with Balance above £10,000 (e.g. £12,000 or £20,000) → follows path 'abdfgh'.

Answer: 2 test cases are needed to achieve 100% statement coverage.

9. Case Study Revisited — CFG & Hybrid Flow Graph for the Gold Card Example

The same gold-card scenario from Section 6 can also be drawn as a control flow graph and a hybrid flow graph — this is exactly the kind of question that could ask you to “draw a control flow graph” or “generate a hybrid flow graph” for a given case study.

- Each of the 4 decision points (gold card?, business class full? [appears twice], economy full?) becomes its own decision node.
- Each decision node is expanded using the if-then-else subgraph pattern — a True edge and a False edge that both eventually rejoin at their own ENDIF node.
- The four ENDIF nodes ultimately connect back together, mirroring how “boarding card” and “bump off flight” were the two possible endings in the original flow chart.
- For the hybrid flow graph version, simply label every edge with a letter — this lets you describe any specific test's route through the case study as a short sequence of letters, exactly like 'abcegh' in the BestInterest example.

Exam tip: when asked to draw this, work outward from the four diamonds — gold card?, business class full?, economy full?, business class full? (second one) — and connect each True/False pair to its own convergence node, exactly following the branch structure listed in Section 6.

10. Quick-Reference Formula Sheet

Coverage (general) = (items exercised ÷ total items) × 100%

Statement Coverage = (statements exercised ÷ total statements) × 100%

Decision Coverage = (decision outcomes exercised ÷ total decision outcomes) × 100%

Remember: 100% decision coverage always implies 100% statement coverage. The reverse is NOT true — you can hit every statement without exercising every branch outcome.

11. Sample Exam Questions (from the lecture slides — practise these)

11. **How many test cases are needed to achieve 100% decision coverage?** → Trace every decision point in the code/diagram and work out the minimum number of paths needed so that each decision has been driven both True and False at least once (see the 5-test airline table in Section 6 as a model answer).
12. **How many test cases would be needed for 100% statement coverage?** → Usually fewer than for decision coverage — find the minimum set of paths that together touch every statement at least once (see the BestInterest example in Section 8).

13. **What level of decision/statement coverage would be achieved by a single input, e.g. A = 10, B = 2? →**

Manually trace that one input through the code, list exactly which statements and which decision outcomes get hit, then apply the formulas in Section 10.

14. **Generate test data for maximum statement coverage.** → Pick the smallest number of inputs whose combined paths cover every statement in the code at least once.

15. **Generate test data for maximum decision coverage.** → Pick inputs so every decision point is driven True at least once and False at least once (or, for a CASE statement, every case outcome is reached at least once).